



《AI大模型Ragent项目》——答案一个字一个字蹦出来流式生成的完整链路

来自： 拿个offer-开源&项目实战

 马丁

2026年05月14日 21:59

开篇引言

上一篇把 Prompt 组装的全过程拆完了——`RAGPromptService.buildStructuredMessages()` 根据场景（KB_ONLY / MCP_ONLY / MIXED）选模板，把 System Prompt、对话历史、检索证据、用户问题按固定骨架拼成一个 `messages` 数组，再加上 `temperature`、`topP`、是否开启深度思考，封装成一个 `ChatRequest`。

到这一步，所有调用参数都就位了。但把 `ChatRequest` 发给大模型，不是一个简单的 HTTP 请求-响应——大模型不会一口气把答案全吐出来，而是一个 token 一个 token 地往外蹦。这些 token 要穿过好几层，才能最终到达用户的浏览器。

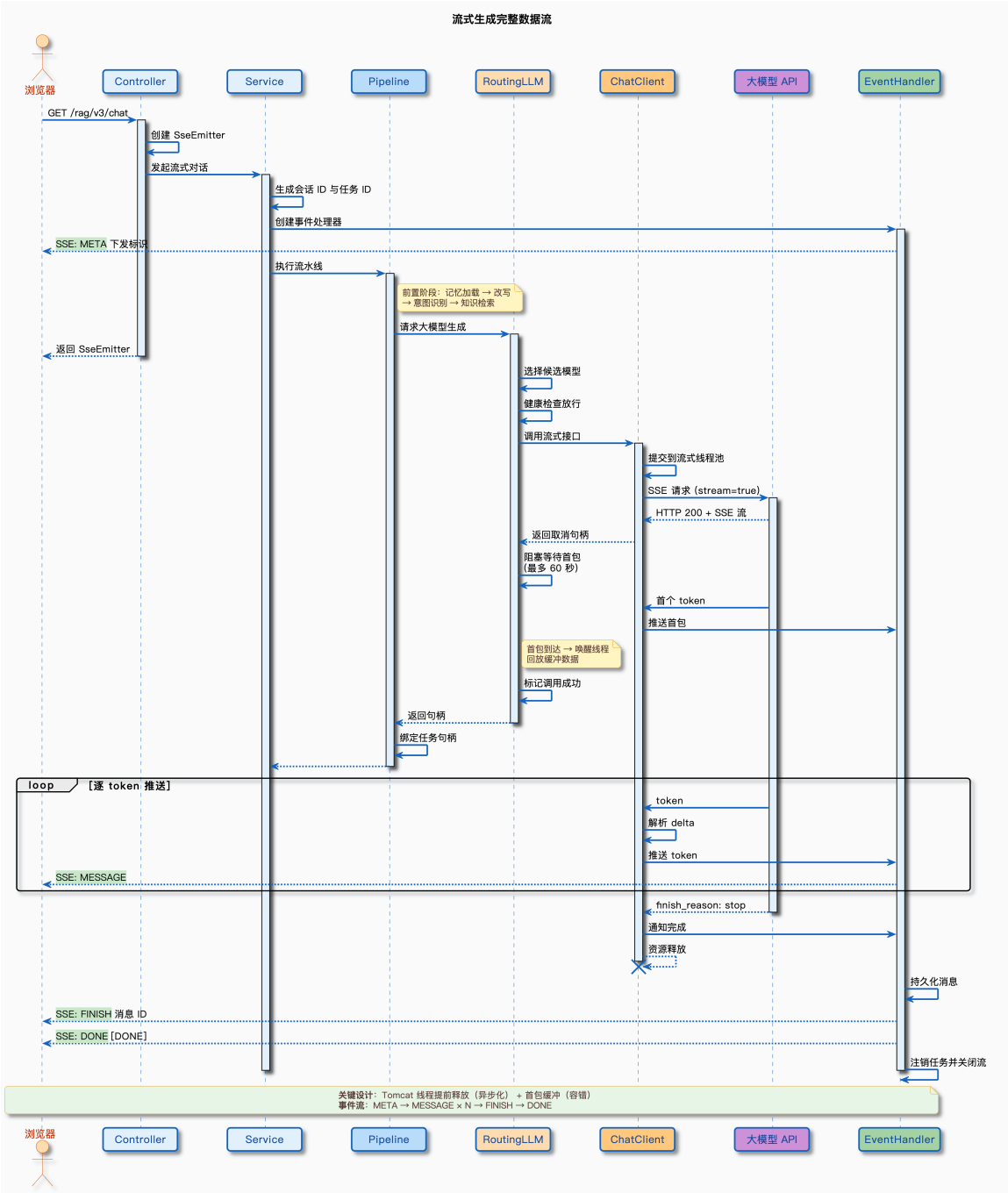
打个比方，这有点像快递的中转流程。你在网上下了单（发出请求），商家一件一件地打包（模型逐 token 生成），然后包裹经过分拣中心、中转站、末端网点（各层处理），最后一件一件送到你家门口（浏览器渲染）。每一层只管自己那段路，接力完成整个投递。

Ragent 的流式生成也是这样，从 Controller 到大模型再到浏览器，中间经过五层接力。本篇就把这条链路从头到尾拆开讲。

全景图：五层接力

1. 一张图看全链路

先上一张时序图，把流式生成的完整数据流画出来。从用户发起请求，到 token 一个个蹦到浏览器，中间经过了哪些参与者、发生了哪些关键事件，一目了然：



2. 五层各自负责什么

整条链路分为五层，每层职责清晰，通过 StreamCallback 接口串联：

层次	关键类	职责	输入 → 输出
入口层	RAGChatController / RAGChatServiceImpl	创建 SSE 连接、生成 ID、创建回调实例	HTTP 请求 → SseEmitter + StreamChatContext
流水线层	StreamChatPipeline	前置阶段 + 组装 Prompt + 调 LLM	StreamChatContext → ChatRequest
调度层	RoutingLLMService / ProbeStreamBridge / ModelHealthStore	选模型、首包探测、容错切换、断路器	ChatRequest → 确定可用模型并启动流式调用
客户端层	AbstractOpenAIStyleChatClient / OpenAIStyleSseParser	OkHttp 读流、SSE 解析、delta 提取、触发回调	HTTP SSE 流 → StreamCallback 回调
推送层	StreamChatEventHandler / SseEmitterSender	缓冲分块、SSE 推送、消息持久化、连接关闭	回调事件 → 浏览器 SSE 事件

这五层的串联方式很朴素——StreamCallback 接口只有四个方法：

```
public interface StreamCallback {  
  
    void onContent(String content);  
  
    default void onThinking(String content) {  
    }  
  
    void onComplete();  
  
    void onError(Throwable error);  
}
```

onContent(String): 正式回答的增量内容, 模型吐一段就回调一次

onThinking(String): 深度思考模式下的思考过程, 和回答内容分开传输, 默认空实现

onComplete(): 整个推理完成, 触发持久化、发事件、关连接

onError(Throwable): 出错了, 通知前端、释放资源

客户端层读到 token 后调 onContent(), 推送层在 onContent() 里把 token 推到浏览器——中间不需要队列、不需要消息总线, 一个接口方法调用就完成了层与层的衔接。

第一层：入口——从 HTTP 请求到 SSE 连接

1. Controller 创建 SseEmitter

```
@GetMapping(value = "/rag/v3/chat", produces = "text/event-stream;charset=UTF-8")  
public SseEmitter chat(@RequestParam String question,  
                       @RequestParam(required = false) String conversationId,  
                       @RequestParam(required = false, defaultValue = "false") Boolean deepThinkin  
                       SseEmitter emitter = new SseEmitter(ragDefaultProperties.getSseTimeoutMs());  
ragChatService.streamChat(question, conversationId, deepThinking, emitter);  
return emitter;  
}
```

三个要点:

produces = "text/event-stream;charset=UTF-8" 告诉 Spring 这是一个 SSE 端点, 响应头会自动带上 Content-Type: text/event-stream

SseEmitter 的超时时间从配置读取, SSE 系列讲过它的核心 API, 这里不再展开

@IdempotentSubmit 注解做了幂等保护——同一用户不能同时发起多个对话

Controller 方法的线程模型很关键: 创建 SseEmitter → 交给 Service 处理 → 立即返回。Tomcat 线程在这里就释放了, 不会被长时间占用。后续的流式推送发生在别的线程上, 这个后面会详细讲。

2. Service 组装上下文

```
@Override  
@ChatRateLimit  
public void streamChat(String question, String conversationId, Boolean deepThinking, SseEmitter em  
    String actualConversationId = StrUtil.isBlank(conversationId) ? IdUtil.getSnowflakeNextIdStr()  
    String taskId = StrUtil.isBlank(RagTraceContext.getTaskId())  
        ? IdUtil.getSnowflakeNextIdStr()  
        : RagTraceContext.getTaskId();  
    boolean thinkingEnabled = Boolean.TRUE.equals(deepThinking);  
  
    StreamCallback callback = callbackFactory.createChatEventHandler(emitter, actualConversationId  
  
    StreamChatContext ctx = StreamChatContext.builder()  
        .question(question)
```

```
        .conversationId(actualConversationId)
        .taskId(taskId)
        .deepThinking(thinkingEnabled)
        .userId(UserContext.getUserId())
        .callback(callback)
        .build();

    try {
        chatPipeline.execute(ctx);
    } catch (Exception e) {
        callback.onError(e);
    }
}
```

这段代码做了四件事：

生成 ID：conversationId 如果前端没传就用 Snowflake 生成一个新的，taskId 优先从 Trace 上下文拿，拿不到再生成。这两个 ID 贯穿整条链路——conversationId 标识一轮对话，taskId 标识一次生成任务。

创建回调实例：StreamCallbackFactory.createChatEventHandler() 把 SseEmitter、两个 ID、以及一堆服务依赖打包传给 StreamChatEventHandler。这个 handler 就是贯穿整条链路的回调实例——从模型调度层到客户端层到推送层，用的都是同一个对象。

```
public StreamCallback createChatEventHandler(SseEmitter emitter,
                                             String conversationId,
                                             String taskId) {
    StreamChatHandlerParams params = StreamChatHandlerParams.builder()
        .emitter(emitter)
        .conversationId(conversationId)
        .taskId(taskId)
        .modelProperties(modelProperties)
        .memoryService(memoryService)
        .conversationGroupService(conversationGroupService)
        .taskManager(taskManager)
        .build();

    return new StreamChatEventHandler(params);
}
```

构建上下文：把问题、ID、用户信息、回调实例封装成 StreamChatContext，交给流水线。

异常兜底：外层 try-catch 包住了整个 pipeline.execute()。流水线的任何阶段抛异常——检索超时、模型调用失败、Prompt 模板渲染出错——都会被这里兜住，通过 callback.onError() 通知前端。

第二层：流水线——Prompt 拼好，调 LLM

1. streamRagResponse 发起流式调用

流水线的前七个阶段（记忆加载、改写、意图识别、检索等）在前面的系列里已经讲完了。本篇聚焦第八个阶段——streamRagResponse()，也就是 Prompt 拼好之后的事情。

```
private void streamRagResponse(StreamChatContext ctx, RetrievalContext retrievalCtx) {
    IntentGroup mergedGroup = intentResolver.mergeIntentGroup(ctx.getSubIntents());

    StreamCancellationHandle handle = streamLLMResponse(
        ctx.getRewriteResult(),
        retrievalCtx,
        mergedGroup,
        ctx.getHistory(),
        ctx.isDeepThinking(),
        ctx.getCallback()
    );
    taskManager.bindHandle(ctx.getTaskId(), handle);
}
```

streamLLMResponse() 里面做了两件事：第一，调上一篇讲的 promptBuilder.buildStructuredMessages() 拼消息数组、构建 ChatRequest；第二，调 llmService.streamChat(chatRequest, callback) 发起流式调用：

```
private StreamCancellationHandle streamLLMResponse(RewriteResult rewriteResult, RetrievalContext c
    IntentGroup intentGroup, List<ChatMessage> hist
    boolean deepThinking, StreamCallback callback)

    PromptContext promptContext = PromptContext.builder()
        .question(rewriteResult.rewrittenQuestion())
        .mcpContext(ctx.getMcpContext())
        .kbContext(ctx.getKbContext())
        .mcpIntents(intentGroup.mcpIntents())
        .kbIntents(intentGroup.kbIntents())
        .intentChunks(ctx.getIntentChunks())
        .build();

    List<ChatMessage> messages = promptBuilder.buildStructuredMessages(
        promptContext, history,
        rewriteResult.rewrittenQuestion(),
        rewriteResult.subQuestions()
    );

    ChatRequest chatRequest = ChatRequest.builder()
        .messages(messages)
        .thinking(deepThinking)
        .temperature(ctx.hasMcp() ? 0.3D : 0D)
        .topP(ctx.hasMcp() ? 0.8D : 1D)
        .build();

    return llmService.streamChat(chatRequest, callback);
}
```

注意返回值——streamChat() 返回一个 StreamCancellationHandle，也就是取消句柄。拿到句柄后，streamRagResponse() 立即通过 taskManager.bindHandle() 把它绑定到当前任务。

2. 取消句柄的绑定时机

你可能会问：为什么不在创建回调时就绑定句柄，非要分两步？

原因是时序问题。回调实例在 Service 层就创建好了（callbackFactory.createChatEventHandler()），那时候连 Prompt 都还没拼，更别提调 LLM 了。取消句柄是 llmService.streamChat() 返回的产物——模型调用开始之后才有句柄可绑。所以 register() 在回调创建时执行（绑定 sender 和取消回调），bindHandle() 在 LLM 调用返回后执行（绑定取消句柄）。

bindHandle() 还有一个防御性设计：如果绑定前已经收到了取消信号（用户手速快，请求刚发出去就点了停止），它会立即调用 handle.cancel() 中断模型调用：

```
public void bindHandle(String taskId, StreamCancellationHandle handle) {
    StreamTaskInfo taskInfo = getOrCreate(taskId);
    taskInfo.handle = handle;
    if (taskInfo.cancelled.get() && handle != null) {
```

```
        handle.cancel();
    }
}
```

取消信号的完整机制——Redis Pub/Sub 广播、跨集群取消、cancelLocal 中断流并推送 CANCEL 事件——下一篇详细讲。

第三层：模型调度——首包探测与容错切换

RoutingLLMService.streamChat() 拿到 ChatRequest 和 StreamCallback 后，不是简单地把请求转发给某个模型，而是带着首包探测、容错切换、断路器三重保护的智能调度。整个过程分三步：

第一步，选候选列表。ModelSelector.selectChatCandidates() 根据是否开启深度思考，返回一个有序的候选模型列表——主模型排前面，备用模型排后面。

第二步，逐个尝试。for 循环遍历候选列表，每个模型先过断路器检查 (healthStore.allowCall())，熔断中的模型直接跳过。通过检查的模型进入首包探测流程。

第三步，首包探测。这是调度层最关键的设计——流式请求不能像同步请求那样简单重试。一旦开始往前端推 token 就没有回头路了，假设主模型吐了三个字就挂了，前端已经收到了那三个字，你没办法撤回再从头来。所以需要在第一个 token 到达之前，先确认模型能不能正常工作。

ProbeStreamBridge 就是干这个的。它站在模型和真正的回调 (StreamChatEventHandler) 之间，经历三个阶段：

1. 缓冲阶段——首包到达前，所有回调事件存入 buffer，不转发给下游。同时通过 CompletableFuture 通知等待的路由主线程探测结果
2. 提交阶段——路由主线程收到 SUCCESS 后调 commit()，把缓冲的事件一次性回放给下游
3. 直通阶段——committed = true 后，新到的事件直接转发，不再缓冲

探测有四种结果：SUCCESS (收到内容，提交缓冲，标记健康)、ERROR (调用出错)、TIMEOUT (60 秒无响应)、NO_CONTENT (正常结束但没吐内容)。后三种都会取消当前连接、标记模型不健康，然后 continue 到下一个候选。如果全部候选失败，通过 callback.onError() 通知前端。

断路器的三态机制 (CLOSED → OPEN → HALF_OPEN)、候选列表的选择算法、首包探测的完整代码和并发细节，在大模型调度引擎实战系列的《三态熔断器与故障转移》和《流式路由的首包探测机制》中已经逐行拆解过，这里不重复展开。

第四层：客户端——读流、解析、回调

探测通过后，实际的流式读取发生在客户端层。这一层的职责是：发起 HTTP 请求 → 逐行读取 SSE 流 → 解析 delta → 触发 StreamCallback 回调。

整个流程由三个组件协作完成：

doStreamChat() 是入口。它构建 OkHttp 请求，通过 StreamAsyncExecutor.submit() 提交到模型流线程池异步执行，立即返回一个 StreamCancellationHandle。如果线程池满了，RejectedExecutionException 兜底——取消 Call、通知回调错误、返回 noop() 句柄。

doStream() 运行在模型流线程上，是实际读取响应的地方。核心是一个 while (!cancelled.get()) 循环，每次迭代调用 source.readUtf8Line() 读一行，交给解析器处理。循环退出有三种情况：流正常结束 (收到 [DONE])、被取消 (cancelled 标志为 true)、流异常中断 (没收到完成信号就 EOF 了)。取消导致的 IOException 是预期行为，不会报给前端。

OpenAIStyleSseParser.parseLine() 负责解析每一行 SSE 数据。它去掉 data: 前缀，识别 [DONE] 结束标记，解析 JSON 提取 choices[0] 中的 content (正式内容) 和 reasoning_content (思考内容，仅深度思考模式)，检查 finish_reason 判断是否完成。解析异常只记日志不中断循环——模型偶尔返回格式不规范的行，不能因为一行解析失败就放弃整个流。

取消机制通过 StreamCancellationHandle 实现，包装了 OkHttp 的 Call 和 AtomicBoolean cancelled 标志，取消时双管齐下：设 cancelled = true 让读循环退出，同时 call.cancel() 中断 HTTP 连接。AtomicBoolean once 保证只执行一次。

这三个组件的完整代码、异步执行模型、SSE 解析的边界情况处理，在大模型调度引擎实战系列的《SSE 流式解析与异步执行》中已经详细讲过，这里只做链路级别的概述。

第五层：推送——从回调到浏览器

1. StreamChatEventHandler 的生命周期

StreamChatEventHandler 实现了 StreamCallback 接口，是 LLM 回调和 SSE 推送之间的桥梁。它的职责是：接收来自客户端层的回调事件 → 分块推送到浏览器 → 完成时持久化消息 → 关闭连接。

1.1 构造与初始化

构造时，StreamChatEventHandler 立即做两件事：

```
private void initialize() {
    sender.sendEvent(SSEEventType.META.value(), new MetaPayload(conversationId, taskId));
}
```

```
taskManager.register(taskId, sender, this::buildCompletionPayloadOnCancel);
}
```

发送 META 事件——前端收到后就知道这个连接对应哪个会话、哪个任务，后续如果要停止生成，就用这个 taskId 调停止接口

在 StreamTaskManager 注册任务——绑定 sender 和取消回调函数。如果此时 Redis 中已经存在取消标记（用户手速极快，在连接建立前就点了停止），register() 会立即发送 CANCEL 事件并关闭连接

1.2 onThinking: 思考内容推送

```
@Override
public void onThinking(String chunk) {
    if (taskManager.isCancelled(taskId)) return;
    if (StrUtil.isBlank(chunk)) return;
    if (thinkingStartMs == 0) {
        thinkingStartMs = System.currentTimeMillis();
    }
    thinking.append(chunk);
    sendChunked(TYPE_THINK, chunk);
}
```

深度思考模式下，模型会先输出一段思考过程，再输出正式回答。onThinking() 处理的就是思考过程：

先检查取消状态，已取消就直接返回

第一次收到 thinking 内容时，记录思考开始时间 thinkingStartMs

累积到 thinking StringBuilder（完成时要持久化）

通过 sendChunked(TYPE_THINK, chunk) 分块推送，前端会把 type = "think" 的内容渲染到思考过程区域

1.3 onContent: 回答内容推送

```
@Override
public void onContent(String chunk) {
    if (taskManager.isCancelled(taskId)) return;
    if (StrUtil.isBlank(chunk)) return;
    if (thinkingStartMs > 0 && thinkingDurationSeconds == 0) {
        thinkingDurationSeconds = Math.max(1,
            Math.round((System.currentTimeMillis() - thinkingStartMs) / 1000.0f));
    }
    answer.append(chunk);
    sendChunked(TYPE_RESPONSE, chunk);
}
```

正式回答内容的处理，和 thinking 类似。多了一个思考计时的逻辑：如果之前有 thinking 阶段（thinkingStartMs > 0），在收到第一个 content 时计算思考耗时。Math.max(1, ...) 保证至少 1 秒——如果模型思考了零点几秒就开始回答，显示 0 秒不太合理。

1.4 onComplete: 收尾动作

```
@Override
public void onComplete() {
    if (taskManager.isCancelled(taskId)) return;
    String messageId = null;
    try {
        String thinkingContent = thinking.isEmpty() ? null : thinking.toString();
        ChatMessage message = ChatMessage.assistant(answer.toString(), thinkingContent, resolveTitleForEvent());
        messageId = memoryService.append(conversationId, userId, message);
    } catch (Exception e) {
        log.error("对话完成时持久化消息失败, conversationId: {}", conversationId, e);
    }
    String title = resolveTitleForEvent();
    sender.sendEvent(SSEEventType.FINISH.value(),
        new CompletionPayload(messageId, title));
}
```

```
sender.sendEvent(SSEEventType.DONE.value(), "[DONE]");
taskManager.unregister(taskId);
sender.complete();
}
```

onComplete() 是整条链路的收尾环节，按顺序做五件事：

1. 持久化消息——把累积的 answer 和 thinking 组装成 ChatMessage.assistant(content, thinkingContent, thinkingDuration)，写入对话记忆。thinking 和 answer 分开存储，持久化失败只记日志不中断流程
2. 发送 FINISH 事件——携带 messageId（前端后续操作如点赞需要）和 title（首次对话时自动生成会话标题）
3. 发送 DONE 事件——[DONE] 是一个信号标记，前端收到后关闭 EventSource 连接
4. 注销任务——从 StreamTaskManager 中移除，同时清理 Redis 中的取消标记
5. 关闭 SSE 连接——sender.complete() 关闭底层的 SseEmitter

1.5 onError：异常处理

```
@Override
public void onError(Throwable t) {
    if (taskManager.isCancelled(taskId)) return;
    taskManager.unregister(taskId);
    sender.fail(t);
}
```

异常处理很简单——注销任务，然后通过 sender.fail() 关闭 SSE 连接。已取消的任务不处理错误，因为取消本身会在别的路径上完善后。

2. 消息分块推送：sendChunked

模型的 onContent() 回调粒度不确定——可能是单个字符，也可能是一整句话。如果每收到一批服务端返回的内容都进行分发，消息一堆一堆的出现，用户感觉不到流式效果。sendChunked() 做了一个缓冲：

```
private void sendChunked(String type, String content) {
    int length = content.length();
    int idx = 0;
    int count = 0;
    StringBuilder buffer = new StringBuilder();
    while (idx < length) {
        int codePoint = content.codePointAt(idx);
        buffer.appendCodePoint(codePoint);
        idx += Character.charCount(codePoint);
        count++;
        if (count >= messageChunkSize) {
            sender.sendEvent(SSEEventType.MESSAGE.value(),
                new MessageDelta(type, buffer.toString()));
            buffer.setLength(0);
            count = 0;
        }
    }
    if (!buffer.isEmpty()) {
        sender.sendEvent(SSEEventType.MESSAGE.value(),
            new MessageDelta(type, buffer.toString()));
    }
}
```

按 messageChunkSize（默认 5 个字符）分块，攒够了推一次 SSE 事件。每个事件的载荷是 MessageDelta(type, delta)，type 区分是回答内容（"response"）还是思考内容（"think"）。

有个细节值得注意：这里用的是 codePointAt() + Character.charCount() 而不是 charAt()。原因是 Java 的 char 是 16 位的，一个 emoji 或者某些生僻字需要两个 char（一个代理对）才能表示。如果用 charAt() 按字符切分，可能把一个 emoji 从中间劈成两半，造成乱码。codePointAt() 按完整的 Unicode 码点遍历，不会出这个问题。

末尾不足一块的内容也会立即推出去，不会丢失。

3. SSE 事件协议设计

前端和后端之间约定了五种 SSE 事件类型：

事件类型	载荷	含义	前端动作
meta	MetaPayload(conversationId, taskId)	连接建立，传递元信息	保存 conversationId 和 taskId
message	MessageDelta(type, delta)	增量内容推送	type="response" 渲染回答，type="think" 渲染思考过程
finish	CompletionPayload(messageId, title)	模型回复完成	更新消息 ID、显示会话标题
done	"[DONE]"	整个流结束	关闭 EventSource 连接
cancel	CompletionPayload(messageId, title)	用户取消生成	同 finish，下一篇详细讲

为什么要分 finish 和 done 两个事件？finish 携带业务数据（消息 ID、标题），前端要据此更新 UI；done 是纯信号，告诉前端可以断开连接了。分开发送可以确保业务数据先到达再断开。

4. SseEmitterSender 的线程安全

SseEmitterSender 封装了 Spring 的 SseEmitter，加了线程安全保护：

```
public class SseEmitterSender {

    private final SseEmitter emitter;
    private final AtomicBoolean closed = new AtomicBoolean(false);

    public SseEmitterSender(SseEmitter emitter) {
        this.emitter = emitter;
        emitter.onCompletion(() -> closed.set(true));
        emitter.onTimeout(() -> closed.set(true));
        emitter.onError(e -> closed.set(true));
    }

    public void sendEvent(String eventName, Object data) {
        if (closed.get()) return;
        try {
            if (eventName == null) {
                emitter.send(data);
                return;
            }
            emitter.send(SseEmitter.event().name(eventName).data(data));
        } catch (Exception e) {
            fail(e);
        }
    }

    public void complete() {
        if (closed.compareAndSet(false, true)) {
            emitter.complete();
        }
    }

    public void fail(Throwable throwable) {
        closeWithError(throwable);
        log.warn("SSE send failed", throwable);
    }

    private void closeWithError(Throwable throwable) {
        if (closed.compareAndSet(false, true)) {
            emitter.close();
        }
    }
}
```

```
        emitter.completeWithError(throwable);
    }
}
}
```

三个保障：

- AtomicBoolean closed + CAS 保证 complete() 和 closeWithError() 的幂等性——多次调用只有第一次生效。这很重要，因为 onComplete() 和 onError() 可能在极端情况下被连续调用
- 三个回调统一标记 closed: onComplete / onTimeout / onError 触发时都把 closed 设为 true。如果客户端断开了连接，onCompletion 会被触发，后续 sendEvent() 就不会再尝试写数据了
- sendEvent() 先检查 closed: 避免向已关闭的连接发送数据导致异常

SSE 系列已经讲过 SseEmitter 的基础用法和注意事项，这里不再展开。

线程模型总览

整条链路涉及三个线程角色，梳理清楚谁在干什么很重要：

线程	角色	生命周期	干什么
Tomcat 线程	处理 HTTP 请求	请求进来 → 返回 SseEmitter	创建 SSE 连接，立即释放，不阻塞
Pipeline / 路由线程	执行前置阶段 + 等首包	Service 调用 → 首包探测完成	加载记忆、改写、意图、检索、拼 Prompt、调 LLM、等首包
模型流线程	读流 + 解析 + 触发回调 + 推 SSE	StreamAsyncExecutor.submit() 提交 → 流读完	doStream() 读 OkHttp 响应流、解析 SSE 行、回调 onContent / onThinking / onComplete, 回调内部执行 sendChunked() → SseEmitter.send()

一个关键点：**SSE 推送发生在模型流线程上**。回调链从 doStream() 开始，经过 ProbeStreamBridge（提交后直通）→ StreamChatEventHandler.onContent() → sendChunked() → SseEmitterSender.sendEvent() → SseEmitter.send(), 整条链路都在同一个线程上，没有额外的线程切换。这意味着模型流线程既负责读流，也负责推 SSE。用一句话概括：**谁读流，谁推送**。这样设计的好处是简单——不需要队列、不需要线程间通信。Spring 的 SseEmitter.send() 本身是线程安全的（由 Servlet 容器保证），所以在模型流线程上调用没有问题。Tomcat 线程在 Controller 返回 SseEmitter 后就释放了，不会被长连接占用。这也是为什么 SSE 不会耗尽 Tomcat 线程池——长时间占用的是 NIO 连接，不是线程。

小结与下一篇预告

本篇把流式生成的完整链路从头到尾拆开了，核心要点：

- 五层接力**——入口层创建 SSE 连接和回调实例，流水线层拼 Prompt 调 LLM，调度层选模型做首包探测和容错切换，客户端层读 OkHttp 流解析 delta 触发回调，推送层分块推 SSE 到浏览器。五层通过 StreamCallback 接口串联，每层只管自己的事
- 调度与客户端**——调度层的首包探测（ProbeStreamBridge 的缓冲→提交→直通三阶段）、三态断路器（ModelHealthStore）、客户端层的 SSE 解析（OpenAIStyleSseParser）和异步执行（StreamAsyncExecutor），这些机制在大模型调度引擎实战系列中已经逐行拆解过，本篇从全链路视角讲清楚它们在五层接力中的位置和衔接方式
- 线程模型**——Tomcat 线程立即释放，Pipeline 线程执行前置阶段并等首包，模型流线程读流、解析、触发回调、推 SSE。谁读流谁推送，不需要额外的推送线程
- 深度思考模式**——从请求体的 enable_thinking 到 SSE 解析的 reasoning_content 提取，到 onThinking() 回调，到 type="think" 事件推送，到持久化时 thinking 和 answer 分开存储，串成一条完整链路
- 取消句柄绑定时机**——回调实例在 Service 层创建，取消句柄在 llmService.streamChat() 返回后才有，taskManager.bindHandle() 分两步绑定并带有防御性设计（绑定前已取消则立即中断）
- 消息分块推送**——sendChunked() 按指定字符（默认 5）分块推送 SSE 事件，用 codePointAt() 正确处理 emoji 等多字节 Unicode 字符
- SSE 事件协议**——META（元信息）→ MESSAGE（增量内容，区分 response 和 think）→ FINISH（消息 ID + 标题）→ DONE（关闭信号），四种事件类型完成前后端协议闭环

本篇讲了 token 怎么从大模型一路流到浏览器，但有一个问题一直在回避——用户如果在生成过程中点了停止生成，会发生什么？这个操作看似简单，背后牵扯的东西却不少：StreamTaskManager 用 Guava Cache 管理本地任务状态，用 Redis Pub/Sub 广播取消信号实现跨集群通知，cancelLocal() 中断 LLM 流、推送 CANCEL 事件、释放资源。在多节点部署的环境下，用户请求可能落在 A 节点，停止请求可能落在 B 节点——取消信号怎么从 B 传到 A？下一篇详细讲。

